

Middleware & Security (Theory)

Welcome to Middleware & Security! As you prepare to connect your React frontend to your FastAPI backend and secure your database credentials for production, understanding server security and middleware is essential. In this session, you will learn how to configure **CORS middleware** in FastAPI and how to securely manage sensitive configuration data using **environment variables** (`.env`).

1. Understanding Middleware in FastAPI

A. What is Middleware?

Middleware is a function that works as a gatekeeper for your application. It sits between the incoming client request and your API route handlers.

- Every incoming request passes through the middleware before reaching your route.
- Every outgoing response passes through the middleware before returning to the client.
- **Common Uses:** Logging request times, handling authentication, adding security headers, and configuring **CORS**.

B. Configuring CORS (Cross-Origin Resource Sharing) in FastAPI

Browsers block requests between different origins (e.g., React at `http://localhost:5173` and FastAPI at `http://localhost:8000`) unless the server explicitly permits it.

To allow your React frontend to communicate with your FastAPI backend without security blocks, you must add `CORSMiddleware` to your FastAPI application instance:

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
```

```
app = FastAPI()
```

```
# Define allowed origins (your React frontend URL)
origins = [
    "http://localhost:5173",
    "http://127.0.0.1:5173",
```

```

]

# Add CORS middleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,          # Allows requests from specified
    allow_credentials=True,         # Allows cookies/authentication
    allow_methods=["*"],            # Allows all HTTP methods (GET, POST,
    allow_headers=["*"],            # Allows all custom headers
)

```

2. Managing Environment Variables (`.env`)

A. The Danger of Hardcoding Credentials

In previous labs, your database connection string looked like this:

```

DATABASE_URL =
"mysql+pymysql://root:your_password@localhost:3306/ml_backend_db"

```

Hardcoding passwords and secret keys directly into your Python code is a major security risk —especially if your code is pushed to public version control repositories like GitHub.

B. Environment Variables & The `.env` File

Environment variables are configuration values stored outside your application code in the operating system environment or within a hidden `.env` file.

- **`.env` File Example:**

```

env
DB_USER=root
DB_PASSWORD=super_secret_password_123
DB_HOST=localhost
DB_PORT=3306
DB_NAME=ml_backend_db

```

C. Loading Variables in Python (`python-dotenv` / Pydantic Settings)

To load these values safely into your application:

1. Install `python-dotenv` : `pip install python-dotenv`
2. Load them inside your configuration file:

```
import os
from dotenv import load_dotenv

# Load variables from .env file into environment
load_dotenv()

db_user = os.getenv("DB_USER")
db_password = os.getenv("DB_PASSWORD")
```

3. Reference Links & Further Reading

- **FastAPI CORS Middleware Guide:** Official documentation on configuring cross-origin resource sharing securely.
 - **Python-Dotenv Documentation:** Guide on managing environment variables in Python projects.
-